

Changes Compared to v4.x

Introduction

Users familiar with version 4.x of the standard may find it helpful to review the key changes introduced in version 5.0, including updates in content, scope, and underlying philosophy.

Of the 286 requirements in version 4.0.3, only 11 remain unchanged, while 15 have undergone minor grammatical adjustments without altering their meaning. In total 109 requirements (38%) are no longer separate requirements in version 5.0 with 50 simply being deleted, 28 removed as duplicates and 31 merged into other requirements. The rest have been revised in some way. Even requirements that were not substantively modified have different identifiers due to reordering or restructuring.

To facilitate adoption of version 5.0, mapping documents are provided to help users trace how requirements from version 4.x correspond to those in version 5.0. These mappings are not tied to release versioning and may be updated or clarified as needed.

Requirement Philosophy

Scope and Focus

Version 4.x included requirements that did not align with the intended scope of the standard; these have been removed. Requirements that did not meet the scope criteria for 5.0 or were not verifiable have also been excluded.

Emphasis on Security Goals Over Mechanisms

In version 4.x, many requirements focused on specific mechanisms rather than the underlying security objectives. In version 5.0, requirements are centered on security goals, referencing particular mechanisms only when they are the sole practical solution, or providing them as examples or supplementary guidance.

This approach recognizes that multiple methods may exist to achieve a given security objective, and avoids unnecessary prescriptiveness that could limit organizational flexibility.

Additionally, requirements addressing the same security concern have been consolidated where appropriate.

Documented Security Decisions

While the concept of documented security decisions may appear new in version 5.0, it is an evolution of earlier requirements related to policy application and threat modeling in version 4.0. Previously, some requirements implicitly demanded analysis to inform the implementation of security controls, such as determining permitted network connections.

To ensure that necessary information is available for implementation and verification, these expectations are now explicitly defined as documentation requirements, making them clear, actionable, and verifiable.

Structural Changes and New Chapters

Several chapters in version 5.0 introduce entirely new content:

- **OAuth and OIDC** –Given the widespread adoption of these protocols for access delegation and single sign-on, dedicated requirements have been added to address the diverse scenarios developers may encounter. This area may eventually evolve into a standalone standard, similar to the treatment of Mobile and IoT requirements in previous versions.
- **WebRTC** –As this technology gains popularity, its unique security considerations and challenges are now addressed in a dedicated section.

Efforts have also been made to ensure that chapters and sections are organized around coherent sets of related requirements.

This restructuring has led to the creation of additional chapters:

- **Self-contained Tokens** –Formerly grouped under session management, self-contained tokens are now recognized as a distinct mechanism and a foundational element for stateless communication (such as in OAuth and OIDC). Due to their unique security implications, they are addressed in a dedicated chapter, with some new requirements introduced in version 5.x.
- **Web Frontend Security** –With the increasing complexity of browser-based applications and the rise of API-only architectures, frontend security requirements have been separated into their own chapter.
- **Secure Coding and Architecture** –New requirements addressing general security practices that did not fit within existing chapters have been grouped here.

Other organizational changes in version 5.0 were made to clarify intent. For example, input validation requirements were moved alongside business logic, reflecting their role in enforcing business rules, rather than being grouped with sanitization and encoding.

The former V1 Architecture chapter has been removed. Its initial section contained requirements that were out of scope, while subsequent sections have been redistributed to relevant chapters, with requirements deduplicated and clarified as necessary.

Removal of Direct Mappings to Other Standards

Direct mappings to other standards have been removed from the main body of the standard. The aim is to prepare a mapping with the OWASP Common Requirement Enumeration (CRE) project, which in turn will link ASVS to a range of OWASP projects and external standards.

Direct mappings to CWE and NIST are no longer maintained, as explained below.

Reduced Coupling with NIST Digital Identity Guidelines

The NIST Digital Identity Guidelines (SP 800-63) have long served as a reference for authentication and authorization controls. In version 4.x, certain chapters were closely aligned with NIST's structure and terminology.

While these guidelines remain an important reference, strict alignment introduced challenges, including less widely recognized terminology, duplication of similar requirements, and incomplete mappings. Version 5.0 moves away from this approach to improve clarity and relevance.

Moving Away from Common Weakness Enumeration (CWE)

The Common Weakness Enumeration (CWE) provides a useful taxonomy of software security weaknesses. However, challenges such as category-only CWEs, difficulties in mapping requirements to a single CWE, and the presence of imprecise mappings in version 4.x have led to the decision to discontinue direct CWE mappings in version 5.0.

Rethinking Level Definitions

Version 4.x described the levels as L1 ("Minimum"), L2 ("Standard"), and L3 ("Advanced"), with the implication that all applications handling sensitive data should meet at least L2.

Version 5.0 addresses several issues with this approach which are described in the following paragraphs.

As a practical matter, whereas version 4.x used tick marks for level indicators, 5.x uses a simple number on all formats of the standard including markdown, PDF, DOCX, CSV, JSON and XML. For backwards compatibility, legacy versions of the CSV, JSON and XML outputs which still use tick marks are also generated.

Easier Entry Level

Feedback indicated that the large number of Level 1 requirements (~120), combined with its designation as the "minimum" level that is not good enough for most applications, discouraged adoption. Version 5.0 aims to lower this barrier by defining Level 1 primarily around first-layer defense requirements, resulting in clearer and fewer requirements at that level. To demonstrate this numerically, in v4.0.3 there were 128 L1 requirements out of a total of 278 requirements, representing 46%. In 5.0.0 there are 70 L1 requirements out of a total of 345 requirements, representing 20%.

The Fallacy of Testability

A key factor in selecting controls for Level 1 in version 4.x was their suitability for assessment through "black box" external penetration testing. However, this approach was not fully aligned with the intent

of Level 1 as the minimum set of security controls. Some users argued that Level 1 was insufficient for securing applications, while others found it too difficult to test.

Relying on testability as a criterion is both relative and, at times, misleading. The fact that a requirement is testable does not guarantee that it can be tested in an automated or straightforward manner. Moreover, the most easily testable requirements are not always those with the greatest security impact or the simplest to implement.

As such, in version 5.0, the level decisions were made primarily based on risk reduction and also keeping in mind the effort to implement.

Not Just About Risk

The use of prescriptive, risk-based levels that mandate a specific level for certain applications has proven to be overly rigid. In practice, the prioritization and implementation of security controls depend on multiple factors, including both risk reduction and the effort required for implementation.

Therefore, organizations are encouraged to achieve the level that they feel like they should be achieving based on their maturity and the message they want to send to their users.

V1 Encoding and Sanitization

Control Objective

This chapter addresses the most common web application security weaknesses related to the unsafe processing of untrusted data. Such weaknesses can result in various technical vulnerabilities, where untrusted data is interpreted according to the syntax rules of the relevant interpreter.

For modern web applications, it is always best to use safer APIs, such as parameterized queries, auto-escaping, or templating frameworks. Otherwise, carefully performed output encoding, escaping, or sanitization becomes critical to the application's security.

Input validation serves as a defense-in-depth mechanism to protect against unexpected or dangerous content. However, since its primary purpose is to ensure that incoming content matches functional and business expectations, requirements related to this can be found in the "Validation and Business Logic" chapter.

V1.1 Encoding and Sanitization Architecture

In the sections below, syntax-specific or interpreter-specific requirements for safely processing unsafe content to avoid security vulnerabilities are provided. The requirements in this section cover the order in which this processing should occur and where it should take place. They also aim to

ensure that whenever data is stored, it remains in its original state and is not stored in an encoded or escaped form (e.g., HTML encoding), to prevent double encoding issues.

#	Description	Level
1.1.1	Verify that input is decoded or unescaped into a canonical form only once, it is only decoded when encoded data in that form is expected, and that this is done before processing the input further, for example it is not performed after input validation or sanitization.	2
1.1.2	Verify that the application performs output encoding and escaping either as a final step before being used by the interpreter for which it is intended or by the interpreter itself.	2

V1.2 Injection Prevention

Output encoding or escaping, performed close to or adjacent to a potentially dangerous context, is critical to the security of any application. Typically, output encoding and escaping are not persisted, but are instead used to render output safe for immediate use in the appropriate interpreter. Attempting to perform this too early may result in malformed content or render the encoding or escaping ineffective.

In many cases, software libraries include safe or safer functions that perform this automatically, although it is necessary to ensure that they are correct for the current context.

#	Description	Level
1.2.1	Verify that output encoding for an HTTP response, HTML document, or XML document is relevant for the context required, such as encoding the relevant characters for HTML elements, HTML attributes, HTML comments, CSS, or HTTP header fields, to avoid changing the message or document structure.	1
1.2.2	Verify that when dynamically building URLs, untrusted data is encoded according to its context (e.g., URL encoding or base64url encoding for query or path parameters). Ensure that only safe URL protocols are permitted (e.g., disallow javascript: or data:).	1
1.2.3	Verify that output encoding or escaping is used when dynamically building JavaScript content (including JSON), to avoid changing the message or document structure (to avoid JavaScript and JSON injection).	1

#	Description	Level
1.2.4	Verify that data selection or database queries (e.g., SQL, HQL, NoSQL, Cypher) use parameterized queries, ORMs, entity frameworks, or are otherwise protected from SQL Injection and other database injection attacks. This is also relevant when writing stored procedures.	1
1.2.5	Verify that the application protects against OS command injection and that operating system calls use parameterized OS queries or use contextual command line output encoding.	1
1.2.6	Verify that the application protects against LDAP injection vulnerabilities, or that specific security controls to prevent LDAP injection have been implemented.	2
1.2.7	Verify that the application is protected against XPath injection attacks by using query parameterization or precompiled queries.	2
1.2.8	Verify that LaTeX processors are configured securely (such as not using the “-shell-escape” flag) and an allowlist of commands is used to prevent LaTeX injection attacks.	2
1.2.9	Verify that the application escapes special characters in regular expressions (typically using a backslash) to prevent them from being misinterpreted as metacharacters.	2
1.2.10	Verify that the application is protected against CSV and Formula Injection. The application must follow the escaping rules defined in RFC 4180 sections 2.6 and 2.7 when exporting CSV content. Additionally, when exporting to CSV or other spreadsheet formats (such as XLS, XLSX, or ODF), special characters (including ‘=’, ‘+’, ‘-’, ‘@’, ‘\t’(tab), and ‘\0’(null character)) must be escaped with a single quote if they appear as the first character in a field value.	3

Note: Using parameterized queries or escaping SQL is not always sufficient. Query parts such as table and column names (including “ORDER BY” column names) cannot be escaped. Including escaped user-supplied data in these fields results in failed queries or SQL injection.

V1.3 Sanitization

The ideal protection against using untrusted content in an unsafe context is to use context-specific encoding or escaping, which maintains the same semantic meaning of the unsafe content but renders it safe for use in that particular context, as discussed in more detail in the previous section.

Where this is not possible, sanitization becomes necessary, removing potentially dangerous characters or content. In some cases, this may change the semantic meaning of the input, but for security

reasons, there may be no alternative.

#	Description	Level
1.3.1	Verify that all untrusted HTML input from WYSIWYG editors or similar is sanitized using a well-known and secure HTML sanitization library or framework feature.	1
1.3.2	Verify that the application avoids the use of <code>eval()</code> or other dynamic code execution features such as Spring Expression Language (SpEL). Where there is no alternative, any user input being included must be sanitized before being executed.	1
1.3.3	Verify that data being passed to a potentially dangerous context is sanitized beforehand to enforce safety measures, such as only allowing characters which are safe for this context and trimming input which is too long.	2
1.3.4	Verify that user-supplied Scalable Vector Graphics (SVG) scriptable content is validated or sanitized to contain only tags and attributes (such as <code>draw</code> graphics) that are safe for the application, e.g., do not contain scripts and <code>foreignObject</code> .	2
1.3.5	Verify that the application sanitizes or disables user-supplied scriptable or expression template language content, such as Markdown, CSS or XSL stylesheets, BBCode, or similar.	2
1.3.6	Verify that the application protects against Server-side Request Forgery (SSRF) attacks, by validating untrusted data against an allowlist of protocols, domains, paths and ports and sanitizing potentially dangerous characters before using the data to call another service.	2
1.3.7	Verify that the application protects against template injection attacks by not allowing templates to be built based on untrusted input. Where there is no alternative, any untrusted input being included dynamically during template creation must be sanitized or strictly validated.	2
1.3.8	Verify that the application appropriately sanitizes untrusted input before use in Java Naming and Directory Interface (JNDI) queries and that JNDI is configured securely to prevent JNDI injection attacks.	2
1.3.9	Verify that the application sanitizes content before it is sent to memcache to prevent injection attacks.	2
1.3.10	Verify that format strings which might resolve in an unexpected or malicious way when used are sanitized before being processed.	2
1.3.11	Verify that the application sanitizes user input before passing to mail systems to protect against SMTP or IMAP injection.	2

#	Description	Level
1.3.12	Verify that regular expressions are free from elements causing exponential backtracking, and ensure untrusted input is sanitized to mitigate ReDoS or Runaway Regex attacks.	3

V1.4 Memory, String, and Unmanaged Code

The following requirements address risks associated with unsafe memory use, which generally apply when the application uses a systems language or unmanaged code.

In some cases, it may be possible to achieve this by setting compiler flags that enable buffer overflow protections and warnings, including stack randomization and data execution prevention, and that break the build if unsafe pointer, memory, format string, integer, or string operations are found.

#	Description	Level
1.4.1	Verify that the application uses memory-safe string, safer memory copy and pointer arithmetic to detect or prevent stack, buffer, or heap overflows.	2
1.4.2	Verify that sign, range, and input validation techniques are used to prevent integer overflows.	2
1.4.3	Verify that dynamically allocated memory and resources are released, and that references or pointers to freed memory are removed or set to null to prevent dangling pointers and use-after-free vulnerabilities.	2

V1.5 Safe Deserialization

The conversion of data from a stored or transmitted representation into actual application objects (deserialization) has historically been the cause of various code injection vulnerabilities. It is important to perform this process carefully and safely to avoid these types of issues.

In particular, certain methods of deserialization have been identified by programming language or framework documentation as insecure and cannot be made safe with untrusted data. For each mechanism in use, careful due diligence should be performed.

#	Description	Level
1.5.1	Verify that the application configures XML parsers to use a restrictive configuration and that unsafe features such as resolving external entities are disabled to prevent XML eXternal Entity (XXE) attacks.	1

#	Description	Level
1.5.2	Verify that deserialization of untrusted data enforces safe input handling, such as using an allowlist of object types or restricting client-defined object types, to prevent deserialization attacks. Deserialization mechanisms that are explicitly defined as insecure must not be used with untrusted input.	2
1.5.3	Verify that different parsers used in the application for the same data type (e.g., JSON parsers, XML parsers, URL parsers), perform parsing in a consistent way and use the same character encoding mechanism to avoid issues such as JSON Interoperability vulnerabilities or different URI or file parsing behavior being exploited in Remote File Inclusion (RFI) or Server-side Request Forgery (SSRF) attacks.	3

References

For more information, see also:

- OWASP LDAP Injection Prevention Cheat Sheet
- OWASP Cross Site Scripting Prevention Cheat Sheet
- OWASP DOM Based Cross Site Scripting Prevention Cheat Sheet
- OWASP XML External Entity Prevention Cheat Sheet
- OWASP Web Security Testing Guide: Client-Side Testing
- OWASP Java Encoding Project
- DOMPurify - Client-side HTML Sanitization Library
- RFC4180 - Common Format and MIME Type for Comma-Separated Values (CSV) Files

For more information, specifically on deserialization or parsing issues, please see:

- OWASP Deserialization Cheat Sheet
- An Exploration of JSON Interoperability Vulnerabilities
- Orange Tsai - A New Era of SSRF Exploiting URL Parser In Trending Programming Languages

V2 Validation and Business Logic

Control Objective

This chapter aims to ensure that a verified application meets the following high-level goals:

- Input received by the application matches business or functional expectations.
- The business logic flow is sequential, processed in order, and cannot be bypassed.

- Business logic includes limits and controls to detect and prevent automated attacks, such as continuous small funds transfers or adding a million friends one at a time.
- High-value business logic flows have considered abuse cases and malicious actors, and have protections against spoofing, tampering, information disclosure, and elevation of privilege attacks.

V2.1 Validation and Business Logic Documentation

Validation and business logic documentation should clearly define business logic limits, validation rules, and contextual consistency of combined data items, so it is clear what needs to be implemented in the application.

#	Description	Level
2.1.1	Verify that the application's documentation defines input validation rules for how to check the validity of data items against an expected structure. This could be common data formats such as credit card numbers, email addresses, telephone numbers, or it could be an internal data format.	1
2.1.2	Verify that the application's documentation defines how to validate the logical and contextual consistency of combined data items, such as checking that suburb and ZIP code match.	2
2.1.3	Verify that expectations for business logic limits and validations are documented, including both per-user and globally across the application.	2

V2.2 Input Validation

Effective input validation controls enforce business or functional expectations around the type of data the application expects to receive. This ensures good data quality and reduces the attack surface. However, it does not remove or replace the need to use correct encoding, parameterization, or sanitization when using the data in another component or for presenting it for output.

In this context, “input” could come from a wide variety of sources, including HTML form fields, REST requests, URL parameters, HTTP header fields, cookies, files on disk, databases, and external APIs.

A business logic control might check that a particular input is a number less than 100. A functional expectation might check that a number is below a certain threshold, as that number controls how many times a particular loop will take place, and a high number could lead to excessive processing and a potential denial of service condition.

While schema validation is not explicitly mandated, it may be the most effective mechanism for full validation coverage of HTTP APIs or other interfaces that use JSON or XML.

Please note the following points on Schema Validation: